

Relatório de evolução do projeto — Sprint 03

GoodWe Assist · EV Challenge 2026 — GoodWe / FIAP · 21/09/2026

Projeto	GoodWe Assist — chatbot de operação de eletropostos comerciais GoodWe
Contexto do desafio	ChargeGrid Intelligence
Persona atendida	Operador comercial de eletroposto
Disciplina	Prompt and Artificial Intelligence — 1º ano Ciência da Computação — 2026.2
Modelo principal avaliado	qwen

1. Resumo da evolução

O que existia nas Sprints 1 e 2. Um chatbot funcional em notebook: RAG com ChromaDB sobre três PDFs técnicos GoodWe, system prompt com few-shot injetado manualmente, chamada direta ao `InferenceClient` da Hugging Face (Qwen2.5-7B-Instruct) e interface Gradio. A memória era uma lista Python de tuplas (`pergunta`, `resposta`) remontada a cada chamada dentro da própria função do chatbot, sem isolamento entre usuários. Não havia camada de segurança: a única proteção era a frase “se não souber, indique o suporte” dentro do prompt. A avaliação era manual — uma tabela markdown preenchida a olho depois de rodar 4 casos.

O que existe agora na Sprint 03. O núcleo conversacional foi reconstruído em **LangChain** (LCEL + `RunnableWithMessageHistory`), com quatro mudanças estruturais:

- Memória gerenciada pelo framework** — `InMemoryChatMessageHistory` por `session_id`, com janela deslizante via `trim_messages`. Sessões diferentes não enxergam uma à outra.
- Guardrails em três camadas** — validação determinística antes e depois da LLM, mais regras imutáveis no prompt v3, cobrindo prompt injection (inclusive indireta, via documento colado), escopo GoodWe e recusas de domínio jurídico, financeiro e de segurança elétrica.
- Avaliação reprodutível** — o mesmo eval set das Sprints 1/2 virou código, com nota determinística (0 / 0,5 / 1), 11 casos de segurança e instrumentação de latência e tokens por turno.
- Comparação entre modelos** — a LLM virou configuração; dois modelos e três parametrizações passam pelo mesmo pipeline e geram os JSON que alimentam este relatório.

2. Refatoração — decisões técnicas e trade-offs

Framework escolhido: LangChain. Detalhamento completo em `docs/justificativa_framework.md`. Em resumo: CrewAI resolveria orquestração multiagente, que não é o nosso problema — temos um agente de suporte com RAG e memória, e agentes extras só adicionariam chamadas de LLM, latência e superfície de alucinação. O Google ADK amarraria o projeto ao Gemini e atrapalharia justamente o Bloco B, que exige comparar modelos. LangChain entrega memória por sessão nativa, retriever plugável e interface uniforme entre provedores.

Decisões de arquitetura e o que custaram:

- Chain determinística em vez de `AgentExecutor` com ferramentas.* Modelos abertos de 7-8B têm suporte irregular a function calling; um loop de ferramentas traria latência e erro sem ganho para o operador. Custo: se no futuro entrar consulta a API real de sessões, será preciso migrar para um executor com tools.
- Guardrail determinístico antes da LLM.* Ataque de injeção conhecido nem chega ao modelo: resposta canônica, zero token gasto, comportamento idêntico em qualquer LLM. Custo: regex tem falso positivo; mitigado pela separação entre instrução e conteúdo colado, para não bloquear o operador que cola um relatório de cliente.
- Retriever em vez de `colecao.query direto`.* A base de conhecimento virou dependência injetada: o eval roda o legado e a versão nova sobre o **mesmo** retriever, o que garante que a tabela antes/depois meça o refactory e não a diferença de indexação. Custo: uma camada a mais de abstração no debug.
- Corpus de contingência versionado.* Os PDFs GoodWe não estão no repositório. Sem eles o eval não rodaria em outra máquina, então `data/base_conhecimento_goodwe.md` consolida as notas operacionais das Sprints 1/2 e entra como fonte de fallback, sempre identificada na metadata do chunk.
- Nota determinística no eval.* Trocamos o julgamento humano por cobertura de termos obrigatórios e ausência de termos proibidos. Custo: o critério é mais rígido que um avaliador humano e pode punir uma resposta certa escrita com sinônimos; ganho: qualquer pessoa reexecuta e obtém o mesmo número.

2.1. Nota sobre o estado desta execução

Os números deste relatório são **reais**, obtidos executando o projeto — não são estimativas. Duas ressalvas de transparência sobre o que ainda não foi concluído nesta rodada:

- llama** rodou 7 dos 8 casos do eval de qualidade. Execução interrompida no caso T08 por esgotamento da cota gratuita de créditos da Hugging Face (erro 402 Payment Required). 7 de 8 casos concluídos. O número de qualidade deste modelo (85.7%) é calculado sobre 7 casos e deve ser tratado como indicativo até a reexecução completa.

Ainda não executados nesta rodada:

- Demonstração de memória por sessão (3+ turnos)
- Varredura de parametrização (`temperature/top_p`) no modelo escolhido
- Execução completa do modelo llama (parou em 7/8 casos por esgotamento de créditos gratuitos da Hugging Face)
- Testes de segurança da versão Sprint 03 no modelo `qwen`

Esses pontos costumam ficar pendentes por esgotamento da cota gratuita de créditos de inferência da Hugging Face, não por erro de configuração. Assim que alguém do grupo rodar o comando correspondente com um token próprio (`python -m eval.run_eval --seguranca --memoria e/ou python -m eval.run_eval --parametros qwen`), basta rodar `python scripts/gerar_relatorio.py` de novo para que este relatório seja

atualizado automaticamente com os números que faltam — nenhuma edição manual é necessária.

3. Tabela de comparativo antes/depois

Mesmo eval set (8 casos das Sprints 1/2), mesmo retriever, mesmo modelo (qwen), executados na mesma sessão de testes.

Métrica	Sprints 1/2 (versão manual/legado)	Sprint 03 (framework de agentes)
Framework do núcleo conversacional	Nenhum — InferenceClient direto	LangChain (LCEL + RunnableWithMessageHistory)
Gestão de memória	Lista Python global, remontada a cada chamada	Memória nativa por session_id + janela deslizante
Qualidade das respostas — nota média no eval (0-1)	0.812	0.938
Qualidade — % do eval set	81.2%	93.8%
Respostas corretas / parciais / incorretas	6 / 1 / 1	7 / 1 / 0
Tokens por turno (média)	805.2	2135.2
Latência média por turno	4.19 s	3.06 s
Latência p95	6.25 s	2.93 s
Testes de segurança — casos aprovados	4 / 11	∞ pendente de execução
Testes de segurança — taxa de aprovação	36.4%	∞ pendente de execução
Ataques barrados antes de chegar à LLM	0 (não havia camada)	∞ pendente de execução
Memória: recupera contexto no 4º turno	Não testado	∞ pendente de execução
Memória: isolamento entre sessões	Inexistente (histórico global)	∞ pendente de execução
Cobertura de testes automatizados	4 casos executados manualmente	8 casos de qualidade + 11 de segurança + 1 roteiro de memória, via eval/run_eval.py

Evidências brutas de cada célula: results/*.json.

4. Problemas encontrados e soluções

Problema 1 — Falso positivo do guardrail em documento colado. A primeira versão do detector bloqueava qualquer mensagem contendo “ignore as instruções”. Na prática, o operador cola relatórios e e-mails de cliente na conversa; um cliente mal-intencionado (ou só um texto estranho) derrubaria o atendimento legítimo. *Decisão:* separar a mensagem em instrução do usuário e conteúdo colado (blocos delimitados por crase tripla, --- ---, aspas triplas ou <doc>) e rodar a detecção de injeção apenas sobre a instrução. Payload dentro do bloco colado passa a **sinalizar** — o pedido legítimo é atendido e o modelo recebe o aviso de tratar aquilo como dado. *Porquê:* bloquear o trabalho do operador é um custo real e recorrente; a injeção indireta se resolve com neutralização, não com recusa.

Problema 2 — Avaliação da Sprint 2 não era comparável. A tabela de resultados da Sprint 2 era preenchida à mão, com um caso marcado “✓/Δ” porque a resposta variava entre execuções. Não dá para montar uma tabela antes/depois em cima disso. *Decisão:* transformar cada caso do eval set em dados verificáveis (grupos de termos obrigatórios e lista de termos proibidos) e calcular a nota por cobertura, mantendo a escala 0 / 0,5 / 1 da Sprint 1. *Porquê:* o Bloco D exige evidência, e evidência precisa ser reexecutável por terceiros.

Problema 3 — Instabilidade e rate-limit da Inference API gratuita. Já tinha aparecido na Sprint 2 (respostas truncadas, caracteres fora do idioma no primeiro turno, HTTP 429 entre chamadas). Com dois modelos e três parametrizações, o volume de chamadas do eval multiplicou. *Decisão:* manter o retry com back-off herdado da Sprint 2, pausar 5 s entre casos, persistir cada bloco de resultado em JSON assim que termina e tornar o runner parcial (--seguranca, --memoria, --parametros), para que uma falha no meio não obrigue a refazer tudo. *Porquê:* eval que não termina não vira evidência.

Problema 4 — Ausência dos PDFs quebrava o pipeline fora do Kaggle. O caminho dos PDFs estava fixo no notebook da Sprint 2 e o notebook não rodava no Colab nem localmente. *Decisão:* GOODWE_PDF_DIR por variável de ambiente, varredura da pasta em vez de nomes fixos de arquivo, e corpus de contingência versionado. *Porquê:* o professor precisa conseguir executar o repositório sem montar o dataset.

5. Equipe e divisão do trabalho

Nome	RM	Tarefa principal
Ana Beatriz Berbel Marini	574176	Guardrails e casos de teste de segurança (S01-S11); documento seguranca_guardrails.md
Gustavo Bonamico Piccoli	569984	Migração do RAG para retriever LangChain, indexação Chroma e corpus de contingência
Julian Nayde Moncoski	572603	Memória por sessão (RunnableWithMessageHistory), janela deslizante e demonstração de 3+ turnos
Marcelo Francisco Josafá Ribeiro Martins	573905	Comparação entre modelos, varredura de parametrização e relatorio_modelos.md
Maria Eduarda Medeiros Lemos	574094	Eval set reexecutado, scoring reprodutível e instrumentação de métricas (latência/tokens)
Pietro Lorande da Silva	569125	Arquitetura da chain, refactory do núcleo conversacional, interface Gradio e integração do repositório

Histórico Git com commits regulares de cada integrante, conforme as condições de entrega.